GREAT SAGE

Graph-Recursive Evidence-Adaptive Tree Search
with Skeptical Adversarial Graph Evaluation

v1.0 — Canonical Implementation Specification

Code Agent Edition | IDE + Sufficient Token Budget

This document is a no-handholding blueprint.
Every section is binding. Every formula is complete. Every field is defined.
Implement exactly as specified; deviations require a recorded ADR amendment.

1. System Foundation & Architectural Decision Records

All ADRs below are binding. Any deviation requires a recorded amendment before implementation. The system targets 32 GB RAM, CPU-only hardware with no GPU assumption.

1.1 Language Boundary: PyO3

Rust is compiled as a thread-safe shared library (.so on Linux / .pyd on Windows) imported by Python via PyO3. Python owns the orchestrator loop and all LLM call sites ("Brain"). Rust owns the graph store, validator, scoring engine, and tool executor ("Harness"). No business logic lives in the Python layer that can be computed deterministically in Rust.

1.2 Concurrency: Tokio + Arc<RwLock<Graph>>

Asynchronous tool execution and parallel Skeptic/Compressor calls are managed by Rust's Tokio runtime. The knowledge graph is stored as:

```
type SharedGraph = Arc<RwLock<petgraph::Graph<NodeKind, EdgeKind>>>;
```

Lock ordering is strictly enforced to prevent deadlock. The canonical acquisition order is:

```
// Canonical lock acquisition order — never invert
1. graph_write   (Phase 4 Compress, Phase 5 Audit, Phase 6 Skeptic mutations)
2. hypothesis_tree_write  (Phase 2 Score update, Phase 8 Reframe)
3. candidate_store_write  (Phase 9 Bank)

// Read locks (non-exclusive, any phase may hold simultaneously)
// graph_read, hypothesis_tree_read, candidate_store_read
```

> **BINDING:** Phases 4, 5, and 6 all mutate the graph. They must execute sequentially within a round, never concurrently. Within-round parallelism is permitted ONLY for independent tool calls inside Phase 3 (Execute), which are read-only with respect to the graph.

1.3 Storage: Petgraph (In-Memory) + Parquet/JSONL Snapshots

Live operations use Rust's petgraph::Graph. Persistence is enforced as follows:

- Snapshot is written after Phase 6 (Skeptic) completes — before any model unload.
- Snapshot format: Parquet for structured tabular export; JSONL for streaming-compatible graph serialization. Implement both; default to JSONL for round-trip fidelity.
- Snapshot must include: full node table, full edge table, current hypothesis tree, current round_id, and all CandidateAnswerNode entries with their lineage UUIDs.
- On restart, the harness deserializes the JSONL snapshot and reconstructs the petgraph in-memory structure before resuming at the next phase.

1.4 Model Serving: Ollama (Local IPC) with Role Swapping

Two logical roles exist: Worker and Skeptic. These map to different model weights. Role swapping (unload / load via Ollama REST API) keeps peak RAM within the 32 GB envelope. The snapshot (§1.3) must be written and fsync'd before initiating a role swap.

Role	Suggested Weight	Active Phases
Worker	Llama-3-8B or Qwen2.5-7B	1 Decompose, 3 Execute, 8 Reframe
Skeptic	Phi-4 or Mistral-7B-Instruct	6 Skeptic, 10 Final Verify
Compressor	Worker (same weights)	4 Compress (reuses loaded Worker)

> *The Compressor runs as a structured prompt to the Worker model. Do not unload/reload for Phase 4. Only swap for Phase 6 and Phase 10.*

1.5 IDE Integration

Great Sage runs as a subprocess launched by the IDE extension (VS Code / JetBrains). Communication is over local Unix domain socket (preferred) or localhost TCP. The IDE extension is responsible for:

- Spawning the Python orchestrator process with the project root as working directory.
- Streaming structured JSON events from the harness to the IDE panel (phase transitions, graph mutations, candidate answer updates).
- Providing the initial task prompt (natural language description + file context) as the Decompose input.
- Exposing a "Run Tests" tool that the harness can invoke as a ToolResult::CodeRun (see §7.2).
- Surfacing the Final Verify result and the winning CandidateAnswerNode to the developer as a diff or patch.

---

2. Canonical Data Schemas

Every node in the system must strictly adhere to these schemas. The Harness Validator (§6) rejects any graph mutation that violates them. All UUIDs are v4. All timestamps are RFC 3339 UTC.

2.1 NodeMeta Header

Required on every node type. Tracks epistemic decay and round provenance.

```
struct NodeMeta {
    id:             Uuid,           // v4, immutable after creation
    created_at:     DateTime<Utc>, // immutable after creation
    last_updated:   DateTime<Utc>, // set by Harness on every mutation
    round_id:       u32,            // round in which this node was created
    belief_score:   f32,            // [0.0, 1.0]; set by Harness scoring only
    source_entropy: f32,            // local uncertainty = 1.0 - belief_score
    is_stale:       bool,           // true if parent hypothesis Refuted
}
```

BINDING: belief_score and source_entropy are computed by the Harness. They must never appear in model output strings. Any model delta that includes these fields is rejected by the Validator.

2.2 HypothesisNode

```
struct HypothesisNode {
    meta:         NodeMeta,
    text:         String,           // falsifiable statement, 1-3 sentences
    status:       HypothesisStatus,
    priority:     f32,              // [0.0, 1.0]; model-proposed, Harness-normalized
    stall_count:  u32,              // incremented by Harness; triggers Reframe at >= 2
    max_reopen:   u8,               // max times this node may reopen after Final
Verify fail; default 2
    reopen_count: u8,               // incremented by Harness on each Final Verify
failure
    test_id:      Option<TestId>,   // Some(id) for test-derived hypotheses (§7.3);
                                    // None for general research hypotheses.
                                    // Harness applies 1.5x EIG boost when Some.
}
```

```
enum HypothesisStatus { Open, Active, Stalled, Resolved, Refuted }
```

2.3 ClaimNode

```
struct ClaimNode {
    meta:         NodeMeta,
    text:         String,
    evidence_ids: Vec<Uuid>,       // must all resolve to EvidenceNode
    status:       ClaimStatus,
}

enum ClaimStatus { Unverified, Supported, Verified, Disputed }
```

support_mass is computed by the Harness as:

```
// support_mass: sum of belief_score of all EvidenceNodes linked to this ClaimNode
fn compute_support_mass(claim: &ClaimNode, graph: &Graph) -> f32 {
    graph.neighbors(claim.meta.id)
        .filter(|n| matches!(n, NodeKind::Evidence(_)))
        .filter(|n| edge_kind(claim.meta.id, n.id) == EdgeKind::Supports)
        .map(|n| n.meta.belief_score)
        .sum()
}
```

2.4 EvidenceNode

```
struct EvidenceNode {
    meta:      NodeMeta,
    snippet:   String,             // raw extracted text; pruned from prompt on
Verified
    source_id: Uuid,               // must resolve to SourceNode
    quality:   QualityTier,
}

enum QualityTier {
    TestVerified,  // confirmed by passing test(s); highest epistemic weight
    Primary,       // direct source, no corroboration required
    Corroborated,  // supported by >= 2 independent sources
    Inferred,      // derived; must carry inference chain in snippet
    Contradicted,  // active Refutes edge present; belief_score -> 0.0
}
```

Harness belief_score assignment by QualityTier:

QualityTier	Initial belief_score	Notes
TestVerified	0.95	Can only reach 1.0 after Skeptic pass + 0 active Refutes
Primary	0.70	Single source, no test confirmation
Corroborated	0.85	Two or more independent root domains
Inferred	0.50	Inference chain must be explicit in snippet
Contradicted	0.00	Set by Harness on Refutes edge creation; immutable until Refutes removed

2.5 SourceNode

```
struct SourceNode {
    meta:         NodeMeta,
    url:          String,          // file path or URL
    status:       SourceStatus,
    domain_auth:  f32,             // from static YAML lookup (see §2.5.1)
    reliability:  f32,             // domain_auth * recency_factor; Harness-computed
}

enum SourceStatus { Unvisited, Visited, Unreachable, Stale }
```

2.5.1 domain_auth Population

domain_auth must NOT be generated by the model (Forbidden Model Writes). Populate from a static YAML file at project root: domain_authority.yaml. Format:

```
# domain_authority.yaml
default: 0.50
overrides:
  stdlib_docs: 0.99
  official_sdk: 0.95
  github.com: 0.75
  stackoverflow.com: 0.65
  local_file: 1.00    # any file:// or relative path
```

recency_factor: for local files, use last-modified timestamp. Files modified within the current session get recency_factor = 1.0. Files older than 30 days get recency_factor = 0.85. Files older than 180 days get 0.70.

2.6 CandidateAnswerNode

```
struct CandidateAnswerNode {
    meta:                NodeMeta,
    content:             String,    // proposed code diff, patch, or answer text
    supporting_claims:   Vec<Uuid>, // all must resolve to ClaimNode
    contradiction_score: f32,       // Harness-computed; see §5.4
    max_reopen_count:    u8,        // default 2; ceiling on Final Verify retry loops
    reopen_count:        u8,        // Harness-incremented
}
```

2.7 ContradictionNode

Created by the Harness on Phase 10 Final Verify failure or Phase 5 Audit structural contradiction. Linked to the demoted CandidateAnswerNode via Invalidates edge.

```
struct ContradictionNode {
    meta:             NodeMeta,
    critique:         String,             // Skeptic output.critique; max 300 chars
    source:           ContradictionSource,
    contested_claims: Vec<Uuid>,          // ClaimNode UUIDs the Skeptic contested
    skeptic_p_fail:   f32,                // p_fail from Skeptic response (see §6.4)
}
```

```
enum ContradictionSource { FinalVerifySkeptic, AuditRefutes }
```

2.8 Edge Kinds

EdgeKind::Proposes connects a HypothesisNode to the CandidateAnswerNode it produced. Round lineage is encoded in NodeMeta.round_id on every node. No RoundNode exists in the graph ontology.

```
enum EdgeKind {
    Supports,       // EvidenceNode   -> ClaimNode, ClaimNode -> HypothesisNode
    Refutes,        // EvidenceNode   -> ClaimNode (contradicting)
    DerivedFrom,    // ClaimNode      -> ClaimNode (inference chain)
    Cites,          // EvidenceNode   -> SourceNode
    Proposes,       // HypothesisNode -> CandidateAnswerNode
    Invalidates,    // ContradictionNode -> CandidateAnswerNode
}
```

---

3. Dual-Layer State Machine

HypothesisStatus controls branch-level execution flow. ClaimStatus controls fact-level epistemic state. These are strictly separated to prevent logic/model mismatch.

3.1 HypothesisStatus Transitions

Status	Transition Event	Precondition	Action
Open	Initialization	Node created by Decompose	Add to pending queue
Active	EIG selection (Phase 2)	EIG > 0.0; budget available	Lock hypothesis for current round
Stalled	Monitor detects EIG drop	stall_count >= 2	Trigger Phase 8 Reframe
Resolved	Phase 5 Audit completes	All leaf claims Verified or Disputed	Release budget; update S_A
Refuted	Phase 6 Skeptic or Phase 10 Final Verify	Skeptic confidence >= delta threshold	Cascade is_stale to all descendant nodes

3.2 ClaimStatus Transitions & Promotion Predicate

Status	Transition Event	Precondition	Notes
Unverified	Creation	ClaimNode inserted by Compressor	No evidence yet
Supported	Phase 4 Compress	>= 1 EvidenceNode with Supports edge	belief_score updated by Harness

Verified	Phase 6 Skeptic pass	Promotion predicate satisfied (see below)	Raw snippet pruned from active prompt
Disputed	Skeptic failure or Refutes edge added	Skeptic result = fail OR active Refutes > 0	belief_score -> 0.0; parent CandidateAnswer penalized

Promotion predicate — a ClaimNode advances to Verified if and only if all four conditions hold simultaneously:

```
fn can_promote(claim: &ClaimNode, skeptic_result: SkepticResult, graph: &Graph) ->
bool {
    let D = diversity_index(claim, graph);
    let support_mass = compute_support_mass(claim, graph);
    let active_refutes = count_active_refutes(claim, graph);

    skeptic_result == SkepticResult::Pass
    && D >= 0.5
    && active_refutes == 0
    && support_mass >= 1.0
}
```

---

4. Operational Phases: The 10-Phase Loop

The Harness enforces phase sequencing. Any model attempt to shortcut phases (e.g., proposing a CandidateAnswerNode during Execute) is rejected by the Validator. Phase transitions are logged to the audit trail with timestamps and round_id.

Routing summary: 1 → 2 → 3 → 4 → 5 → 6 → 7. From 7: Continue → 2, Stall → 8 → 2, Converge → 9 → 10. From 10: Pass → emit, Fail+reopen → 3 (micro-round), Fail+ceiling → emit with uncertainty.

Phase 1: Decompose [STANDARD]

Worker generates a falsifiable Hypothesis Tree from the task prompt. Each leaf hypothesis must be testable in <= 5 tool steps. For code tasks, Decompose is preceded by a mandatory Spec Pass (see §7.3).

Harness:

- Validate that model output is a valid JSON hypothesis tree against HypothesisNode schema.
- Assign initial priority values: normalize model-proposed priorities to [0.0, 1.0] range.
- Insert all HypothesisNodes into the graph with status = Open.
- Set round_id = 0 for all nodes created in this phase.

LLM (Worker or Skeptic):

- Receive: task prompt + (for code tasks) formal spec, test suite, and current failing test list.
- Produce: JSON hypothesis tree. Each node contains: text (falsifiable claim), priority (float), children (array). Leaf nodes have no children.
- Do NOT produce: belief_score, source_entropy, is_stale, or any NodeMeta fields.

Code Agent: Before generating the hypothesis tree, produce the Spec Pass output (§7.3). The hypothesis tree leaf nodes must map 1:1 to test IDs in the generated test suite.

Phase 2: Score [NEW]

Harness deterministically ranks Open hypothesis branches by Expected Information Gain (EIG) and allocates per-branch token budget before any LLM call fires. No LLM involvement.

Harness:

- Compute EIG for each Open HypothesisNode using the formula: EIG(h) = (1.0 - h.meta.belief_score) * h.priority
- Sort Open nodes descending by EIG.
- Discard nodes where EIG < 0.05 (negligible gain threshold).
- Allocate token budget per branch: budget(h) = total_round_budget * (EIG(h) / sum_EIG_all_active)
- Mark selected branches as Active.
- Log EIG scores and budget allocations to audit trail.

Lock: Acquire: hypothesis_tree_write. Release before Phase 3 begins.

Code Agent: For code tasks: EIG of a hypothesis tied to a failing test is boosted by * 1.5. This ensures failing tests always get disproportionate budget until they pass.

Phase 3: Execute [ENHANCED]

Worker performs bounded search within the allocated token budget for each Active branch. Maximum 5 tool steps per branch per round. Independent branches may execute in parallel (read-only w.r.t. the graph).

Harness:

- Enforce 5-tool-step ceiling per branch; terminate branch on breach and mark Stalled.
- Track token consumption per branch against allocated budget from Phase 2.
- Collect all ToolResult objects and queue for Phase 4.
- Do NOT mutate the graph during Execute. All graph writes are deferred to Phase 4.
- For parallel branches: spawn Tokio tasks, collect futures, await all before proceeding to Phase 4.

LLM (Worker or Skeptic):

- Receive: active hypothesis text + structured prompt template (see §8.1) + graph summary of already-Verified claims.
- For each tool call: output a ToolCall struct (see §7.2). Do not produce graph mutations directly.
- Produce: list of ToolResult objects. Each must include: tool_type, raw_output, and inferred_claim_texts (list of strings for Compressor extraction).

Lock: No graph lock held during Execute. Parallel branches are safe.

Code Agent: ToolResult::CodeRun is a first-class tool type (see §7.2). Test pass/fail counts from CodeRun results directly determine QualityTier = TestVerified for extracted claims.

Phase 4: Compress [ENHANCED]

Compressor (Worker model with structured prompt) extracts claim/evidence deltas from raw ToolResults and proposes graph mutations. Harness validates and applies all mutations atomically.

Harness:

- Acquire graph_write lock.

- Resolve tmp_* refs: assign new UUIDv4 to each tmp_claim_N, tmp_evidence_N, tmp_source_N in the batch. Replace all tmp_* occurrences in edge from_id/to_id with the assigned UUIDs. Reject any edge whose resolved from_id or to_id does not exist in the graph or in the current batch.
- For each proposed ClaimNode: validate against ClaimNode schema. Reject any node with belief_score, source_entropy, or Verification_Status fields in the model output.
- For each proposed EvidenceNode: verify source_id resolves to an existing SourceNode or create a new SourceNode (domain_auth from YAML lookup, never from model).
- Compute support_mass for each new ClaimNode. Set initial ClaimStatus = Supported if support_mass > 0, else Unverified.
- Apply all validated mutations atomically (all-or-nothing per batch).
- Release graph_write lock.

LLM (Worker or Skeptic):

- Receive: raw ToolResult list + compression prompt template (see §8.2).
- Produce: JSON delta object containing: new_claims (list), new_evidence (list), new_sources (list), new_edges (list). No belief_score or status fields.

Lock: Acquire graph_write. Release before Phase 5.

Code Agent: For CodeRun results: create EvidenceNodes with quality = TestVerified for each passing test. Create EvidenceNodes with quality = Inferred for stderr analysis. Map test IDs to hypothesis IDs for direct linkage.

Phase 5: Audit [NEW]

Harness traverses the graph and performs deterministic contradiction detection. No LLM involvement. Purely structural graph analysis.

Harness:

- Acquire graph_write lock.
- Contradiction check: for each ClaimNode pair (A, B) where both have Supports edges to the same HypothesisNode, check if A and B share a root domain in their SourceNode lineage. If so, flag as potentially corroborated (not independent).
- Direct contradiction check: traverse all Refutes edges. For each Refutes(E -> C): if E.meta.belief_score >= 0.7, set C.status = Disputed and C.meta.belief_score = 0.0.
- Inference chain validation: for each DerivedFrom(C1 -> C2) edge, verify C1.status != Unverified. If C1 is Unverified, set C2.status = Disputed.
- Stale cascade: for each HypothesisNode with status = Refuted, set is_stale = true on all descendant nodes.
- Append Audit summary to round log (contradiction count, stale count, disputed count).
- Release graph_write lock.

Lock: Acquire graph_write. Release before Phase 6.

Phase 6: Skeptic [NEW]

Adversarial model attacks each Supported ClaimNode before promotion to Verified. Role swap: load Skeptic weights. Skeptic result feeds the Harness promotion predicate.

Harness:

- Serialize graph snapshot BEFORE initiating role swap (see §1.3). fsync required.
- Trigger Ollama role swap: unload Worker, load Skeptic.
- For each Supported ClaimNode: pass to Skeptic with adversarial prompt template (see §8.3).
- Collect SkepticResult (Pass | Fail | Inconclusive) for each claim.

- Acquire graph_write lock.
- Apply promotion predicate (§3.2) to each Supported claim. Promote to Verified or Disputed accordingly.
- For Verified claims: remove raw snippet text from active prompt context (O(1) prompt growth enforcement).
- Release graph_write lock.
- Trigger Ollama role swap: unload Skeptic, load Worker.

LLM (Worker or Skeptic):

- Receive: claim text + supporting evidence snippets + adversarial prompt (§8.3).
- Produce: JSON with fields: result (Pass|Fail|Inconclusive), confidence (float 0-1), critique (string, max 200 chars). No graph fields.

Lock: Acquire graph_write after Skeptic responses collected. Release before Phase 7.

Code Agent: Skeptic receives the full failing-test output for any claim derived from CodeRun results. A claim that "the function handles edge case X" is attacked with the actual test failure message for that case.

Phase 7: Monitor [ENHANCED]

Harness evaluates meta-state of the current round. Three possible outcomes: Continue (loop to Phase 2), Stall/Reframe (go to Phase 8), Converge (go to Phase 9). No LLM involvement.

Harness:

- Stall check: for each Active HypothesisNode, if EIG(h) in this round < 0.05 OR EIG delta < 0.01 vs. prior round, increment h.stall_count. If h.stall_count >= 2, mark h.status = Stalled.
- Redundancy check: if >= 60% of tool calls in this round retrieved content already represented in an existing Verified ClaimNode, increment global redundancy_counter.
- Convergence check: if all HypothesisNodes are Resolved or Refuted (none remain Open/Active/Stalled), set convergence_flag = true. CandidateAnswerNode scoring happens in Phase 9 and is NOT available here; do not attempt to read candidate scores in Phase 7.
- Budget check: if total_tokens_consumed >= total_budget, force convergence_flag = true regardless of scores.
- Routing: if convergence_flag -> Phase 9. Else if any Stalled hypotheses -> Phase 8. Else -> Phase 2 (next round).

Code Agent: For code tasks: convergence_flag may also be triggered if all_tests_pass == true. This is the primary termination condition.

Phase 8: Reframe [NEW]

Triggered when >= 1 HypothesisNode is Stalled. Prunes the dead branch and re-seeds exploration from the current graph state. Two-step process: Harness structural prune, then Worker re-decomposition.

Harness:

- Acquire hypothesis_tree_write lock.
- For each Stalled HypothesisNode: set status = Refuted. Cascade is_stale to descendants.
- Extract reframe seed: collect all ClaimNodes with status = Supported (not yet Verified) + all Unverified ClaimNodes + all Disputed ClaimNodes with active Refutes edges. Serialize as structured JSON summary.
- Release hypothesis_tree_write lock.

- Pass reframe seed + original task prompt to Worker with reframe prompt template (see §8.4).
- Receive new HypothesisNode proposals from Worker. Validate and insert with status = Open.
- Return to Phase 2.

LLM (Worker or Skeptic):

- Receive: reframe prompt (§8.4) containing the current graph state summary — what is known, what has been falsified, what is disputed.
- Produce: JSON list of new HypothesisNode proposals. Same schema as Decompose output. Must NOT reproduce hypotheses that are already Refuted or Resolved.
- Constraint: new hypotheses must be structurally distinct from the pruned hypotheses (Harness checks by text embedding similarity; threshold cosine distance > 0.3).

Lock: hypothesis_tree_write during prune step. Released before Worker call.

Code Agent: For code tasks: the reframe seed always includes the full list of currently failing tests. New hypotheses must address specific test failures, not re-attempt already-Resolved implementations.

Phase 9: Bank [ENHANCED]

Consolidate Verified claims into CandidateAnswerNodes. Compute Global Candidate Score S_A for each candidate. Select the winner by S_A.

Harness:

- Acquire candidate_store_write lock.
- Group Verified ClaimNodes by the HypothesisNode they support. Each Resolved HypothesisNode produces one CandidateAnswerNode.
- Compute S_A for each candidate (see §5.2).
- Rank candidates by S_A descending.
- Release candidate_store_write lock.
- Proceed to Phase 10 with the top-ranked CandidateAnswerNode.

LLM (Worker or Skeptic):

- No LLM call in Phase 9. Purely Harness arithmetic.

Lock: candidate_store_write. Release before Phase 10.

Code Agent: For code tasks: CandidateAnswerNode.content must be a well-formed diff/patch, not prose. The Harness validates that the content is parseable as unified diff format before banking.

Phase 10: Final Verify [NEW]

High-temperature adversarial stress-test on the winning CandidateAnswerNode before emission. Role swap to Skeptic. Includes termination guard to prevent infinite reopen loops.

Harness:

- Serialize graph snapshot (pre-swap).
- Role swap to Skeptic weights.
- Pass winning CandidateAnswerNode.content + all supporting claims + Final Verify prompt (see §8.5).
- Collect adversarial critique and confidence score.
- Confidence semantics (binding): the Skeptic's confidence field always means p_fail — the probability that the candidate answer is WRONG. A result of Pass with confidence=0.8 means "I am 80% sure this is wrong, but my overall judgment is Pass." That is incoherent; enforce: if result=Pass, p_fail must be < delta. If result=Fail, p_fail must be >= delta. The Harness rejects any Skeptic response where result and p_fail contradict the delta threshold.

- PASS path (Skeptic p_fail < delta=0.35): emit CandidateAnswerNode.content as final output.
- FAIL path (Skeptic p_fail >= 0.35): check reopen_count against max_reopen_count. If reopen_count < max_reopen_count: increment reopen_count, set CandidateAnswerNode.meta.belief_score = 0.0, create ContradictionNode(source=FinalVerifySkeptic, critique=Skeptic.critique, contested_claims=Skeptic.contested_claim_ids, skeptic_p_fail=Skeptic.confidence), attach Invalidates edge ContradictionNode -> CandidateAnswerNode, re-open parent HypothesisNode, role-swap back to Worker, return to Phase 3 for one targeted micro-round. If reopen_count >= max_reopen_count: emit best available candidate with uncertainty annotation.
- Role swap back to Worker (or terminate if done).

LLM (Worker or Skeptic):

- Receive: final verify prompt (§8.5) with full candidate content and claim lineage.
- Produce: JSON with: result (Pass|Fail), confidence (float), contested_claim_ids (list of UUIDs), critique (string).

> **Code Agent:** For code tasks: Final Verify includes actually running the full test suite via ToolResult::CodeRun. A Skeptic text-critique alone is insufficient. The Harness checks: all_tests_pass. If false, this is an automatic FAIL regardless of Skeptic confidence score.

---

5. Controller Mathematics & Scoring Formulas

All formulas are implemented in Rust. No formula computation may be delegated to the LLM. All inputs to formulas are graph-resident fields set by the Harness.

5.1 Expected Information Gain: EIG(h)

```
// EIG for a HypothesisNode h
// Used in Phase 2 for branch selection and budget allocation
fn eig(h: &HypothesisNode) -> f32 {
    (1.0 - h.meta.belief_score) * h.priority
}

// Code agent variant: boost EIG for hypotheses tied to failing tests
fn eig_code(h: &HypothesisNode, has_failing_test: bool) -> f32 {
    let base = (1.0 - h.meta.belief_score) * h.priority;
    if has_failing_test { base * 1.5 } else { base }
}

// Budget allocation across all Active branches in a round
// total_round_budget: token count allocated for this round
fn allocate_budget(nodes: &[HypothesisNode], total_round_budget: u32) -> HashMap<Uuid,
u32> {
    let eigs: Vec<f32> = nodes.iter().map(|h| eig(h)).collect();
    let sum_eig: f32 = eigs.iter().sum();
    nodes.iter().zip(eigs.iter()).map(|(h, e)| {
        (h.meta.id, (total_round_budget as f32 * e / sum_eig) as u32)
    }).collect()
}
```

5.2 Diversity Index: D(S)

```
// D(S): fraction of unique root domains across all SourceNodes
// supporting a given ClaimNode. Measures source independence.
// Used in promotion predicate (§3.2) and S_A formula (§5.3)
fn diversity_index(claim: &ClaimNode, graph: &Graph) -> f32 {
    let sources: Vec<&SourceNode> = get_evidence_sources(claim, graph);
    if sources.is_empty() { return 0.0; }
    let unique_domains: HashSet<String> = sources
        .iter()
        .map(|s| extract_root_domain(&s.url))
        .collect();
    unique_domains.len() as f32 / sources.len() as f32
}
```

5.3 Global Candidate Score: S_A

```
// S_A: score for a CandidateAnswerNode
// Higher is better. Penalized by unresolved gaps, contradictions, and
contradiction_score.
const EPSILON: f32 = 1e-6;
const LAMBDA: f32 = 1.5;
const MU: f32 = 2.0;    // penalty weight for contradiction_score

fn score_candidate(candidate: &mut CandidateAnswerNode, graph: &Graph) -> f32 {
    let support_claims: Vec<&ClaimNode> = resolve_claims(&candidate.supporting_claims,
graph);

    // W_support: sum of belief_scores of all Verified supporting claims
    let w_support: f32 = support_claims.iter()
        .filter(|c| c.status == ClaimStatus::Verified)
        .map(|c| c.meta.belief_score)
        .sum();

    // D: diversity index averaged across all supporting claims
    let d: f32 = if support_claims.is_empty() { 0.0 } else {
        support_claims.iter().map(|c| diversity_index(c, graph)).sum::<f32>()
            / support_claims.len() as f32
    };

    // W_refute: sum of belief_scores of all Disputed claims in lineage
    let w_refute: f32 = support_claims.iter()
        .filter(|c| c.status == ClaimStatus::Disputed)
        .map(|c| c.meta.belief_score)
        .sum();

    // Unresolved_Gaps: count of Disputed or Unverified claims in support lineage
    let unresolved_gaps: u32 = support_claims.iter()
        .filter(|c| matches!(c.status, ClaimStatus::Disputed |
ClaimStatus::Unverified))
        .count() as u32;

    // contradiction_score: computed here and stored on the node (see §5.4)
    let cs = compute_contradiction_score(candidate, &support_claims);
    candidate.contradiction_score = cs;

    (w_support * d) / (EPSILON + w_refute + LAMBDA * unresolved_gaps as f32 + MU * cs)
}
```

5.4 Contradiction Score: compute_contradiction_score

contradiction_score measures what fraction of a candidate's claim lineage is disputed, weighted by evidence strength. It is computed by score_candidate (§5.3) and stored on CandidateAnswerNode. Higher = more contested.

```
// contradiction_score for a CandidateAnswerNode
// Range: [0.0, 1.0]. 0.0 = no disputed claims. 1.0 = all claims disputed.
fn compute_contradiction_score(
    candidate: &CandidateAnswerNode,
    support_claims: &[&ClaimNode],
) -> f32 {
    if support_claims.is_empty() { return 0.0; }
    let disputed_mass: f32 = support_claims.iter()
        .filter(|c| c.status == ClaimStatus::Disputed)
        .map(|c| c.meta.belief_score)
        .sum();
    let total_mass: f32 = support_claims.iter()
        .map(|c| c.meta.belief_score)
        .sum();
    // Returns 0.0 if no claims have any belief mass (avoid div-by-zero)
    disputed_mass / (total_mass + EPSILON)
}
```

5.5 Belief Score vs. Support Mass: Explicit Distinction

These are two separate quantities serving different purposes. Do not conflate them.

Quantity	Definition	Used in
belief_score (on ClaimNode)	Scalar [0,1]. Initialized to the QualityTier initial value of the best linked EvidenceNode (max-tier prior). Updated multiplicatively by Bayesian update after Skeptic pass. Represents epistemic confidence in the claim.	promotion predicate (must be >= D threshold), S_A w_support sum, contradiction_score
support_mass (computed)	Additive sum of belief_score across ALL EvidenceNodes linked to a ClaimNode via Supports edges. Represents the total weight of accumulated evidence, not the quality of the best piece.	promotion predicate (must be >= 1.0), used as the threshold check for Verified promotion

Rationale: support_mass >= 1.0 requires that the total evidence weight clears a meaningful threshold, preventing promotion on a single weak piece of evidence (e.g., one Inferred node with belief_score=0.5 alone is insufficient). belief_score being max-initialized ensures a single TestVerified result drives a claim to 0.95 confidence quickly.

5.6 Belief Score Updates

belief_score for a ClaimNode is updated by the Harness after every Phase 4 (Compress) and Phase 6 (Skeptic) run. Formula:

```
// After Compress: update belief_score from QualityTier of linked EvidenceNodes
fn update_belief_from_evidence(claim: &mut ClaimNode, graph: &Graph) {
    let best_tier_score = get_evidence_nodes(claim, graph)
        .iter()
        .map(|e| tier_initial_score(&e.quality))
```

```
        .fold(0.0f32, f32::max);
    claim.meta.belief_score = best_tier_score;
    claim.meta.source_entropy = 1.0 - best_tier_score;
}

// After Skeptic Pass: apply Bayesian update
// skeptic_confidence: model-output confidence that claim is CORRECT (post-pass)
fn bayesian_update(claim: &mut ClaimNode, skeptic_confidence: f32) {
    // Simple multiplicative update; prior = current belief_score
    claim.meta.belief_score = (claim.meta.belief_score * skeptic_confidence)
        .min(1.0).max(0.0);
}
```

---

6. Deterministic Validator Catalog

The Rust Harness executes all checks in this catalog before any graph mutation is applied. A single failing check causes the entire mutation batch to be rejected (all-or-nothing). Rejected mutations are logged to the audit trail with the failing check ID and the offending model output.

Check ID	Rule	Action on Failure
V-01	Provenance Integrity: no ClaimNode without at least one EvidenceNode parent via Supports edge	Reject batch; log V-01
V-02	Reference Integrity: all UUID references in new nodes resolve to existing graph nodes	Reject batch; log V-02 with dangling UUID
V-03	No-Write Zone: id and created_at fields must not appear in model delta output	Reject batch; log V-03
V-04	Forbidden Model Writes: belief_score, source_entropy, Verification_Status, domain_auth, reliability must not appear in model delta	Reject batch; log V-04
V-05	Context Pruning: when ClaimNode.status transitions to Verified, remove raw snippet from active prompt context immediately	Apply mutation; trigger pruning
V-06	Hypothesis Shortcut: model may not propose CandidateAnswerNode during Execute phase	Reject node; log V-06
V-07	Reframe Similarity: new hypotheses from Phase 8 must have cosine distance > 0.3 from all Refuted hypotheses	Reject node; prompt Worker for alternative
V-08	Stale Write: no Supports edge may be added to a node where is_stale = true	Reject edge; log V-08
V-09	Reopen Ceiling: CandidateAnswerNode.reopen_count must not exceed max_reopen_count	Block reopen; emit with uncertainty annotation
V-10	Schema Completeness: every new node must carry a fully populated NodeMeta header (all fields present)	Reject node; log V-10 with missing fields
V-11	CodeRun Patch Validity: CandidateAnswerNode.content must parse as valid unified diff before banking	Reject banking; log V-11
V-12	support_mass minimum: a Supported ClaimNode must have support_mass >= 0.01 (epsilon guard against zero-weight evidence)	Downgrade to Unverified; log V-12

7. Code Agent Extension

This section defines the modifications that transform Great Sage from a research agent into a code agent. These are additive — all base schemas and phases remain unchanged. The extensions slot into the existing structure at defined injection points.

7.1 QualityTier: TestVerified

TestVerified is a first-class QualityTier above Primary. It represents a claim confirmed by a passing automated test. This is the highest-confidence evidence tier available to the code agent.

- A ClaimNode may only receive TestVerified evidence if the linked EvidenceNode carries a test_ids field referencing one or more passing test IDs from a ToolResult::CodeRun.
- Initial belief_score for TestVerified EvidenceNode: 0.95. It may reach 1.0 only after Skeptic pass with confidence >= 0.95 and zero active Refutes edges.
- A ClaimNode with all evidence at TestVerified tier and support_mass >= 1.0 skips the Skeptic adversarial prompt and goes directly to Harness promotion (Skeptic result is implicitly Pass).

7.2 Tool Type Extension: ToolResult::CodeRun

ToolResult is extended with a CodeRun variant. This is the primary evidence-gathering mechanism for the code agent.

```
enum ToolCall {
    Search { query: String },
    Visit  { url: String },
    CodeRun {
        command:    String,         // shell command to execute
        working_dir: String,        // relative to project root
        timeout_secs: u32,          // default 30; Harness enforced
    },
}

enum ToolResult {
    Search { snippets: Vec<String> },
    Visit  { content: String },
    CodeRun {
        stdout:       String,
        stderr:       String,
        exit_code:    i32,
        tests_passed: Vec<TestId>,  // empty if no test runner detected
        tests_failed: Vec<TestId>,  // empty if all pass
        duration_ms:  u32,
    },
}

type TestId = String;              // e.g. "test_module::test_edge_case_empty_input"
```

The Harness maps ToolResult::CodeRun to graph mutations:

- For each test_id in tests_passed: create EvidenceNode(quality=TestVerified, snippet=stdout excerpt for that test).
- For each test_id in tests_failed: create EvidenceNode(quality=Inferred, snippet=failure message). Add Refutes edge to the hypothesis that predicted this test would pass.
- exit_code != 0 with empty tests_passed/tests_failed (compile error): create EvidenceNode(quality=Inferred, snippet=stderr). Set linked HypothesisNode.stall_count += 1.

7.3 Code Agent Spec Pass (Phase 1 Injection)

Before the Worker generates the hypothesis tree in Phase 1, a mandatory Spec Pass runs. This is a structured LLM call that produces the formal specification and test suite. The hypothesis tree is then generated FROM the spec, not from the raw task prompt.

Spec Pass Output Schema

```
struct SpecPass {
    language:          String,
    entry_point:       String,      // function/module signature
    input_contract:    String,      // types, ranges, preconditions
    output_contract:   String,      // types, postconditions
    invariants:        Vec<String>, // runtime invariants
    edge_cases:        Vec<String>, // must be addressed by test suite
    performance_req:   Option<String>, // e.g. "O(n log n) sort"
    generated_tests:   Vec<TestSpec>,
}

struct TestSpec {
    id:          TestId,
    description: String,
    test_code:   String,            // runnable test in target language
    maps_to_hyp: Option<String>,    // hypothesis text this test validates
}
```

Spec Pass to Hypothesis Tree Mapping

The Harness generates initial HypothesisNode entries directly from the TestSpec list:

```
// For each TestSpec, create a corresponding HypothesisNode
fn spec_to_hypothesis(spec: &TestSpec) -> HypothesisNode {
    HypothesisNode {
        meta: NodeMeta::new(round_id=0, belief_score=0.0),
        text: format!("The implementation satisfies test: {}", spec.description),
        status: HypothesisStatus::Open,
        priority: 1.0,             // all test hypotheses start at max priority
        stall_count: 0,
        max_reopen: 2,
        reopen_count: 0,
    }
}
```

The Worker may also add additional hypotheses beyond the test-derived ones (e.g., "The implementation uses the correct algorithm"). Test-derived hypotheses are distinguished by a test_id field that non-test hypotheses lack. The Harness gives these a 1.5× EIG boost (§5.1).

7.4 Primary Convergence Condition

For code agents, Phase 7 Monitor has an additional convergence trigger that supersedes the S_A threshold check:

```
// Code agent convergence: all tests pass
fn check_convergence_code(last_coderun: &ToolResult::CodeRun) -> bool {
    last_coderun.tests_failed.is_empty()
        && last_coderun.exit_code == 0
        && !last_coderun.tests_passed.is_empty()
}
```

When this condition is true, Phase 10 Final Verify MUST still run the test suite (not just the Skeptic adversarial prompt). A text-passing Final Verify with failing tests is treated as a FAIL.

---

8. Prompt Templates

Templates use {VARIABLE} notation. Variables in CAPS are injected by the Harness. Do not modify the structural directives (OUTPUT FORMAT, CONSTRAINT). The model must produce exactly the specified JSON schema or the Validator rejects the output.

8.1 Execute Prompt (Worker, Phase 3)

```
SYSTEM: You are a code research agent. You gather evidence by calling tools.
You do not write to the knowledge graph. You do not propose final answers.
Your only output is a list of ToolCall objects and inferred claim texts.

TASK: {ORIGINAL_TASK_PROMPT}

CURRENT HYPOTHESIS: {HYPOTHESIS_TEXT}

ALREADY VERIFIED (do not re-investigate these):
{VERIFIED_CLAIMS_SUMMARY}

KNOWN FAILING TESTS (prioritize investigating these):
{FAILING_TEST_IDS_AND_MESSAGES}

TOOL BUDGET: {TOOL_BUDGET} tool calls maximum.
TOKEN BUDGET: {TOKEN_BUDGET} tokens maximum for this branch.

OUTPUT FORMAT (JSON only, no prose):
{
  "tool_calls": [
    {"type": "CodeRun", "command": "...", "working_dir": ".", "timeout_secs": 30},
    {"type": "Search", "query": "..."},
    {"type": "Visit", "url": "..."}
  ],
  "inferred_claim_texts": ["claim 1", "claim 2"]
}
```

8.2 Compress Prompt (Worker/Compressor, Phase 4)

```
SYSTEM: You are a knowledge extraction agent. Extract structured claims and evidence
```

```
from raw tool results. You do not set belief_score, source_entropy, or status fields.
The Harness computes all numeric scores.

RAW TOOL RESULTS:
{TOOL_RESULT_LIST_JSON}

CURRENT HYPOTHESIS CONTEXT: {HYPOTHESIS_TEXT}

EXISTING GRAPH NODE IDs YOU MAY REFERENCE (for edges to existing nodes):
{CURRENT_GRAPH_SUMMARY_IDS}

Use tmp_claim_N / tmp_evidence_N / tmp_source_N for NEW nodes in this batch.
Use real UUIDs from CURRENT_GRAPH_SUMMARY_IDS for edges to existing nodes.

// IMPORTANT: you cannot know the UUIDs of nodes created in the same batch.
// Use temporary local refs (tmp_claim_N, tmp_evidence_N, tmp_source_N).
// The Harness resolves all tmp_* refs to real UUIDs during atomic commit.
// You may also reference existing graph UUIDs from CURRENT_GRAPH_SUMMARY.

OUTPUT FORMAT (JSON only, no prose):
{
  "new_claims": [
    {
      "tmp_id": "tmp_claim_1",
      "text": "...",
      "inferred_from_test": "test_id_or_null"
    }
  ],
  "new_evidence": [
    {
      "tmp_id": "tmp_evidence_1",
      "snippet": "...",
      "source_url": "...",
      "quality_hint": "Primary|Inferred|TestVerified"
    }
  ],
  "new_edges": [
    {
      "from_id": "tmp_evidence_1",
      "to_id": "tmp_claim_1",
      "kind": "Supports|Refutes|DerivedFrom|Cites"
    }
  ]
}

CONSTRAINT: Use tmp_* refs for all nodes created in this batch.
Do not include belief_score, source_entropy, status, domain_auth,
or reliability in any output field. The Harness will reject the batch if present.
```

8.3 Skeptic Prompt (Skeptic model, Phase 6)

```
SYSTEM: You are an adversarial auditor. Your goal is to find weaknesses in claims.
You are skeptical. You look for: insufficient evidence, single-source dependence,
logical leaps, unverified assumptions, and edge cases not covered.

CLAIM TO EVALUATE: {CLAIM_TEXT}

SUPPORTING EVIDENCE:
{EVIDENCE_SNIPPETS_WITH_SOURCES}
```

```
FOR CODE CLAIMS — FAILING TEST OUTPUT (if applicable):
{FAILING_TEST_OUTPUT}

OUTPUT FORMAT (JSON only):
{
  "result": "Pass|Fail|Inconclusive",
  "confidence": 0.0-1.0,
  "critique": "max 200 chars; specific objection or null if Pass"
}

CONSTRAINT: result="Pass" means the claim is sufficiently supported.
confidence is your confidence that your result is correct, not the claim confidence.
Do not propose graph mutations, belief scores, or node IDs.
```

8.4 Reframe Prompt (Worker, Phase 8)

```
SYSTEM: You are a research strategist. A previous investigation branch has stalled.
You must propose new, structurally distinct hypotheses to explore.

ORIGINAL TASK: {ORIGINAL_TASK_PROMPT}

CURRENT GRAPH STATE:
  Verified claims: {VERIFIED_CLAIMS_COUNT}
  Disputed claims: {DISPUTED_CLAIMS_COUNT}
  Refuted hypotheses: {REFUTED_HYPOTHESES_TEXT_LIST}
  Still failing tests: {FAILING_TEST_IDS}
  Unresolved gaps: {UNRESOLVED_GAPS_SUMMARY}

The following hypotheses have been REFUTED and must not be reproposed:
{REFUTED_HYPOTHESES_TEXT_LIST}

OUTPUT FORMAT (JSON only):
{
  "new_hypotheses": [
    {"text": "falsifiable claim", "priority": 0.0-1.0, "rationale": "short string"}
  ]
}

CONSTRAINT: Each new hypothesis must be structurally distinct from refuted ones.
For code tasks: each hypothesis must address a specific failing test by test_id.
```

8.5 Final Verify Prompt (Skeptic model, Phase 10)

```
SYSTEM: You are a final review auditor. This is the last check before output.
Be maximally critical. Find any flaw that would cause the answer to be wrong or
the code to fail in production.

CANDIDATE ANSWER:
{CANDIDATE_ANSWER_CONTENT}

FULL SUPPORTING CLAIM LINEAGE:
{ALL_SUPPORTING_CLAIMS_WITH_EVIDENCE}

FOR CODE: FULL TEST RUN OUTPUT:
{FULL_TEST_SUITE_OUTPUT}
```

```
OUTPUT FORMAT (JSON only):
{
  "result": "Pass|Fail",
  "confidence": 0.0-1.0,   // p_fail: probability candidate is WRONG.
                             // Pass requires confidence < 0.35.
                             // Fail requires confidence >= 0.35.
                             // Harness rejects contradictory pairs (e.g. Pass + 0.8).
  "contested_claim_ids": ["uuid1", "uuid2"],
  "critique": "specific objection, max 300 chars"
}

CONSTRAINT: confidence is ALWAYS p_fail (probability the answer is wrong).
result=Pass means p_fail < 0.35. result=Fail means p_fail >= 0.35.
For code tasks: if any test shows FAILED in the test run output,
output result=Fail with confidence >= 0.35 regardless of other assessment.
```

8.6 Spec Pass Prompt (Worker, Phase 1 Code Injection)

```
SYSTEM: You are a software specification engineer. Before any implementation,
produce a complete formal specification and a minimal but sufficient test suite.
The test suite will be used to drive the entire implementation loop.

TASK: {ORIGINAL_TASK_PROMPT}
TARGET LANGUAGE: {LANGUAGE}
EXISTING CODE CONTEXT:
{RELEVANT_FILE_CONTENTS}

OUTPUT FORMAT (JSON only):
{
  "language": "...",
  "entry_point": "function_name(args) -> return_type",
  "input_contract": "...",
  "output_contract": "...",
  "invariants": ["..."],
  "edge_cases": ["..."],
  "performance_req": "... or null",
  "generated_tests": [
    {
      "id": "test_module::test_name",
      "description": "what this test validates",
      "test_code": "full runnable test code",
      "maps_to_hyp": "hypothesis text this test proves or null"
    }
  ]
}

CONSTRAINT: Every edge_case listed must have at least one corresponding test.
Tests must be runnable as-is; no placeholder assertions.
```

---

9. Local Deployment, Configuration & Evaluation Thresholds

9.1 Directory Structure

```
great_sage/
├── harness/                # Rust crate (library)
```

```
│   ├── src/
│   │   ├── graph.rs       # petgraph wrapper, SharedGraph type
│   │   ├── validator.rs   # V-01 through V-12
│   │   ├── scorer.rs      # EIG, D, S_A, belief_score updates
│   │   ├── phases.rs      # phase state machine, routing logic
│   │   ├── snapshot.rs    # JSONL/Parquet serialization
│   │   └── tools.rs       # ToolCall/ToolResult execution
│   └── Cargo.toml
├── orchestrator/          # Python package
│   ├── __init__.py
│   ├── brain.py           # LLM call sites (Worker, Skeptic, Compressor)
│   ├── ollama_client.py   # role swap, model load/unload
│   ├── loop.py            # main 10-phase loop, calls harness via PyO3
│   └── prompts.py         # §8 template rendering
├── domain_authority.yaml  # §2.5.1
├── config.yaml            # see §9.2
└── snapshots/             # JSONL round snapshots
```

9.2 config.yaml Reference

```
model:
  worker:          llama3:8b        # Ollama model tag
  skeptic:         phi4:latest
  worker_ctx:      8192             # context window tokens
  skeptic_ctx:     4096

budget:
  total_tokens:    200000           # per full run
  round_tokens:    25000            # per round (Phase 2 distributes)
  max_rounds:      12
  tool_steps:      5                # max per branch per round (Phase 3)
  tool_timeout:    30               # seconds (CodeRun)

thresholds:
  eig_min:         0.05             # discard branches below this
  eig_delta_stall: 0.01             # stall detection delta
  stall_count:     2                # reframe trigger
  convergence_sa:  0.85             # S_A threshold for convergence
  final_verify_delta: 0.35          # Skeptic confidence threshold (Fail if >=)
  diversity_min:   0.50             # D threshold in promotion predicate
  support_mass_min: 1.00            # support_mass threshold
  reopen_max:      2                # max Final Verify reopens per candidate
  redundancy_pct:  0.60             # redundancy_counter trigger
  reframe_cosine:  0.30             # minimum cosine distance for new hypotheses

snapshot:
  format:          jsonl            # jsonl or parquet
  path:            ./snapshots/

ide:
  socket_type:     unix             # unix or tcp
  socket_path:     /tmp/great_sage.sock
  event_stream:    true
```

9.3 Evaluation & Success Thresholds

Metric	Target	Measurement

Stall detection	< 2 redundant tool steps before reframe	redundancy_counter per round
Prompt stability	Active prompt size ± 15% of Round 1 size after Round 4	token count logged per phase
O(1) prompt growth	Zero raw snippets from Verified claims in active prompt	Validator V-05 enforcement
Final Verify pass rate	> 90% of runs pass on first Final Verify attempt	reopen_count distribution
Convergence rate	> 80% of runs converge before max_rounds	Phase 7 routing log
Test pass rate (code)	100% tests_passed on Final Verify CodeRun	Phase 10 enforcement

9.4 Audit Trail

Every phase transition, graph mutation, Validator rejection, and LLM call is logged to an append-only JSONL audit trail at snapshots/audit_{run_id}.jsonl. Log entries must include: timestamp, round_id, phase_id, event_type, node_ids_affected, token_count_delta, and error_code (null if no error). This log is the primary debugging tool and must not be truncated during a run.